

Complete Data Structures and Algorithms Guide

DSA Reference

Implementation all data structure and algorithms that I have come across in C++/Python. Some of them are through solving problems and some of them the direct implemetation of pseudo code.

Table of contents

1	Introduction	3
2	Linear Data Structures	3
2.1	Array	3
2.1.1	C++ Implementation	4
2.1.2	Python Implementation	5
2.2	Linked List	6
2.2.1	C++ Implementation	7
2.2.2	Python Implementation	9
2.3	Stack	10
2.3.1	C++ Implementation	11
2.3.2	Python Implementation	13
2.4	Queue	13
2.4.1	C++ Implementation	15
2.4.2	Python Implementation	16
3	Non-Linear Data Structures	17
3.1	Binary Tree	17
3.1.1	C++ Implementation	18
3.1.2	Python Implementation	21
3.2	Binary Search Tree (BST)	22
3.2.1	C++ Implementation	24
3.2.2	Python Implementation	26

3.3	Hash Table	28
3.3.1	C++ Implementation	30
3.3.2	Python Implementation	32
3.4	Graph	33
3.4.1	C++ Implementation (Adjacency List)	34
3.4.2	Python Implementation	36
4	Sorting Algorithms	38
4.1	Bubble Sort	38
4.1.1	C++ Implementation	39
4.1.2	Python Implementation	40
4.2	Selection Sort	40
4.2.1	C++ Implementation	42
4.2.2	Python Implementation	42
4.3	Insertion Sort	43
4.3.1	C++ Implementation	45
4.3.2	Python Implementation	45
4.4	Merge Sort	45
4.4.1	C++ Implementation	47
4.4.2	Python Implementation	48
4.5	Quick Sort	49
4.5.1	C++ Implementation	51
4.5.2	Python Implementation	51
4.6	Heap Sort	52
4.6.1	C++ Implementation	54
4.6.2	Python Implementation	55
5	Searching Algorithms	56
5.1	Linear Search	56
5.1.1	C++ Implementation	57
5.1.2	Python Implementation	58
5.2	Binary Search	58
5.2.1	C++ Implementation	60
5.2.2	Python Implementation	61
6	Graph Algorithms	62
6.1	Breadth-First Search (BFS)	62
6.1.1	C++ Implementation	63
6.1.2	Python Implementation	64
6.2	Depth-First Search (DFS)	64
6.2.1	C++ Implementation	65
6.2.2	Python Implementation	67

6.3	Dijkstra's Algorithm	67
6.3.1	C++ Implementation	68
6.3.2	Python Implementation	69
7	Dynamic Programming	70
7.1	Dynamic Programming Fundamentals	70
7.2	Fibonacci with DP	71
7.2.1	C++ Implementation	71
7.2.2	Python Implementation	72
8	String Algorithms	73
8.1	KMP Algorithm (Pattern Matching)	73
8.1.1	C++ Implementation	73
8.1.2	Python Implementation	75
9	Advanced Data Structures	76
9.1	Trie (Prefix Tree)	76
9.1.1	C++ Implementation	77
9.1.2	Python Implementation	79
9.2	Segment Tree	80
9.2.1	C++ Implementation	80
9.2.2	Python Implementation	82
10	Conclusion	83

1 Introduction

This comprehensive guide covers fundamental data structures and algorithms with detailed explanations and implementations in both C++ and Python. Each section includes theoretical concepts, complexity analysis, advantages, disadvantages, and complete working code.

2 Linear Data Structures

2.1 Array

Definition

A collection of elements stored in contiguous memory locations, accessed by index.

Detailed Explanation: Arrays are the most fundamental data structure where elements of the same type are stored in consecutive memory locations. Each element can be accessed

directly using its index, making random access very efficient. The index typically starts from 0 in most programming languages.

Key Characteristics

- Fixed size (in static arrays)
- Homogeneous elements (same data type)
- Contiguous memory allocation
- Zero-based indexing

Advantages

- Fast access to elements $O(1)$
- Memory efficient (no extra memory for pointers)
- Cache friendly due to locality of reference
- Simple implementation

Disadvantages

- Fixed size (static arrays)
- Insertion and deletion are expensive $O(n)$
- Memory waste if array is not fully utilized

Time Complexity

- Access: $O(1)$
- Search: $O(n)$
- Insertion: $O(n)$
- Deletion: $O(n)$

2.1.1 C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;

class Array {
private:
    vector<int> arr;
    int capacity;
```

```

public:
    Array(int size) : capacity(size) {
        arr.reserve(size);
    }

    void insert(int element) {
        if (arr.size() < capacity) {
            arr.push_back(element);
        }
    }

    void remove(int index) {
        if (index >= 0 && index < arr.size()) {
            arr.erase(arr.begin() + index);
        }
    }

    int get(int index) {
        if (index >= 0 && index < arr.size()) {
            return arr[index];
        }
        return -1;
    }

    void display() {
        for (int val : arr) {
            cout << val << " ";
        }
        cout << endl;
    }
};

```

2.1.2 Python Implementation

```

class Array:
    def __init__(self, capacity):
        self.arr = []
        self.capacity = capacity

    def insert(self, element):

```

```

        if len(self.arr) < self.capacity:
            self.arr.append(element)

    def remove(self, index):
        if 0 <= index < len(self.arr):
            self.arr.pop(index)

    def get(self, index):
        if 0 <= index < len(self.arr):
            return self.arr[index]
        return None

    def display(self):
        print(self.arr)

```

2.2 Linked List

Definition

A linear data structure where elements are stored in nodes, each containing data and a reference to the next node.

Detailed Explanation: A linked list consists of nodes where each node contains data and a pointer/reference to the next node in the sequence. Unlike arrays, linked list elements are not stored in contiguous memory locations. The list is accessed through the head node, and traversal happens by following the links from one node to the next.

Types of Linked Lists

- **Singly Linked List:** Each node points to the next node
- **Doubly Linked List:** Each node has pointers to both next and previous nodes
- **Circular Linked List:** Last node points back to the first node

Key Characteristics

- Dynamic size
- Non-contiguous memory allocation
- Sequential access only
- Each node contains data and pointer(s)

Advantages

- Dynamic size - can grow/shrink during runtime
- Efficient insertion/deletion at the beginning $O(1)$
- Memory efficient - allocates memory as needed
- No memory waste

Disadvantages

- No random access - must traverse from head
- Extra memory overhead for storing pointers
- Not cache friendly due to non-contiguous memory
- No backward traversal in singly linked lists

Time Complexity

- Access: $O(n)$
- Search: $O(n)$
- Insertion: $O(1)$ at head, $O(n)$ at arbitrary position
- Deletion: $O(1)$ at head, $O(n)$ at arbitrary position

2.2.1 C++ Implementation

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;

    Node(int value) : data(value), next(nullptr) {}
};

class LinkedList {
private:
    Node* head;

public:
    LinkedList() : head(nullptr) {}
};
```

```

void insertAtHead(int data) {
    Node* newNode = new Node(data);
    newNode->next = head;
    head = newNode;
}

void insertAtTail(int data) {
    Node* newNode = new Node(data);
    if (!head) {
        head = newNode;
        return;
    }

    Node* current = head;
    while (current->next) {
        current = current->next;
    }
    current->next = newNode;
}

void deleteNode(int data) {
    if (!head) return;

    if (head->data == data) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }

    Node* current = head;
    while (current->next && current->next->data != data) {
        current = current->next;
    }

    if (current->next) {
        Node* temp = current->next;
        current->next = current->next->next;
        delete temp;
    }
}

```



```

void display() {
    Node* current = head;
    while (current) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "NULL" << endl;
}

~LinkedList() {
    while (head) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}
};

```

2.2.2 Python Implementation

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_head(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_tail(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        return

```

```

        current = self.head
        while current.next:
            current = current.next
        current.next = new_node

def delete_node(self, data):
    if not self.head:
        return

    if self.head.data == data:
        self.head = self.head.next
        return

    current = self.head
    while current.next and current.next.data != data:
        current = current.next

    if current.next:
        current.next = current.next.next

def display(self):
    current = self.head
    while current:
        print(current.data, end=" -> ")
        current = current.next
    print("None")

```

2.3 Stack

Definition

A LIFO (Last In, First Out) data structure where elements can only be added or removed from the top.

Detailed Explanation: A stack is a linear data structure that follows the LIFO principle, meaning the last element added is the first one to be removed. Think of it like a stack of plates - you can only add or remove plates from the top. The stack has a “top” pointer that indicates the position of the most recently added element.

Key Operations

- **Push:** Add an element to the top
- **Pop:** Remove and return the top element
- **Peek/Top:** View the top element without removing it
- **isEmpty:** Check if stack is empty

Key Characteristics

- LIFO ordering
- Access restricted to top element only
- Can be implemented using arrays or linked lists
- Automatic memory management in recursion

Advantages

- Simple implementation
- Efficient push/pop operations $O(1)$
- Automatic handling of function calls and recursion
- Memory efficient for temporary storage

Disadvantages

- Limited access - only top element accessible
- No random access to elements
- Can lead to stack overflow if not managed properly
- Fixed size in array implementation

Time Complexity

- Push: $O(1)$
- Pop: $O(1)$
- Peek/Top: $O(1)$

2.3.1 C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;
```

```

class Stack {
private:
    vector<int> stack;

public:
    void push(int data) {
        stack.push_back(data);
    }

    void pop() {
        if (!isEmpty()) {
            stack.pop_back();
        }
    }

    int top() {
        if (!isEmpty()) {
            return stack.back();
        }
        return -1;
    }

    bool isEmpty() {
        return stack.empty();
    }

    int size() {
        return stack.size();
    }

    void display() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            cout << stack[i] << endl;
        }
    }
};

```

2.3.2 Python Implementation

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, data):
        self.stack.append(data)

    def pop(self):
        if not self.is_empty():
            return self.stack.pop()
        return None

    def top(self):
        if not self.is_empty():
            return self.stack[-1]
        return None

    def is_empty(self):
        return len(self.stack) == 0

    def size(self):
        return len(self.stack)

    def display(self):
        for item in reversed(self.stack):
            print(item)
```

2.4 Queue

Definition

A FIFO (First In, First Out) data structure where elements are added at the rear and removed from the front.

Detailed Explanation: A queue is a linear data structure that follows the FIFO principle, similar to a real-world queue or line. Elements are added (enqueued) at the rear/back and removed (dequeued) from the front. It maintains two pointers: front (for removal) and rear (for insertion).

Types of Queues

- **Simple Queue:** Basic FIFO queue
- **Circular Queue:** Rear connects back to front when space is available
- **Priority Queue:** Elements have priorities, highest priority dequeued first
- **Double-ended Queue (Deque):** Insertion and deletion possible at both ends

Key Operations

- **Enqueue:** Add element to the rear
- **Dequeue:** Remove element from the front
- **Front:** Get the front element without removing
- **isEmpty:** Check if queue is empty
- **isFull:** Check if queue is full (in fixed-size implementations)

Key Characteristics

- FIFO ordering
- Access restricted to front and rear
- Can be implemented using arrays, linked lists, or circular arrays
- Fair processing - first come, first served

Advantages

- Fair scheduling (FIFO)
- Efficient insertion and deletion $O(1)$
- Natural representation of real-world queues
- Useful for breadth-first processing

Disadvantages

- No random access to middle elements
- Fixed size in array implementation
- Memory waste in simple array implementation
- Cannot access elements in the middle

Time Complexity

- Enqueue: $O(1)$
- Dequeue: $O(1)$
- Front: $O(1)$

2.4.1 C++ Implementation

```
#include <iostream>
using namespace std;

class Queue {
private:
    int* arr;
    int front, rear, capacity, size;

public:
    Queue(int cap) : capacity(cap), front(0), rear(-1), size(0) {
        arr = new int[capacity];
    }

    void enqueue(int data) {
        if (size == capacity) return;
        rear = (rear + 1) % capacity;
        arr[rear] = data;
        size++;
    }

    void dequeue() {
        if (isEmpty()) return;
        front = (front + 1) % capacity;
        size--;
    }

    int getFront() {
        if (!isEmpty()) {
            return arr[front];
        }
        return -1;
    }

    bool isEmpty() {
        return size == 0;
    }

    bool isFull() {
        return size == capacity;
    }
}
```

```

    }

    void display() {
        if (isEmpty()) return;
        for (int i = 0; i < size; i++) {
            cout << arr[(front + i) % capacity] << " ";
        }
        cout << endl;
    }

    ~Queue() {
        delete[] arr;
    }
};

```

2.4.2 Python Implementation

```

from collections import deque

class Queue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, data):
        self.queue.append(data)

    def dequeue(self):
        if not self.is_empty():
            return self.queue.popleft()
        return None

    def front(self):
        if not self.is_empty():
            return self.queue[0]
        return None

    def is_empty(self):
        return len(self.queue) == 0

    def size(self):

```



```

        return len(self.queue)

    def display(self):
        print(list(self.queue))

```

3 Non-Linear Data Structures

3.1 Binary Tree

Definition

A tree data structure where each node has at most two children (left and right).

Detailed Explanation: A binary tree is a hierarchical data structure where each node has at most two children, referred to as left child and right child. The topmost node is called the root, and nodes with no children are called leaves. Each node contains data and pointers to its left and right children.

Tree Terminology

- **Root:** Topmost node with no parent
- **Parent:** Node that has children
- **Child:** Node that has a parent
- **Leaf:** Node with no children
- **Height:** Length of longest path from root to leaf
- **Depth:** Length of path from root to a specific node
- **Level:** All nodes at the same depth

Types of Binary Trees

- **Full Binary Tree:** Every node has 0 or 2 children
- **Complete Binary Tree:** All levels filled except possibly the last, which is filled left to right
- **Perfect Binary Tree:** All internal nodes have 2 children, all leaves at same level
- **Balanced Binary Tree:** Height difference between left and right subtrees ≤ 1

Tree Traversal Methods

- **Inorder (Left-Root-Right):** Visits left subtree, root, then right subtree
- **Preorder (Root-Left-Right):** Visits root, left subtree, then right subtree

- **Postorder (Left-Right-Root):** Visits left subtree, right subtree, then root
- **Level-order:** Visits nodes level by level (BFS)

Key Characteristics

- Hierarchical structure
- Each node has at most two children
- Recursive definition
- Non-linear data structure

Advantages

- Hierarchical data representation
- Efficient searching in ordered trees
- Natural recursive structure
- Dynamic size

Disadvantages

- No guarantee of balance (can become skewed)
- More complex than linear structures
- Pointer overhead
- Not cache friendly

Time Complexity

- Search: $O(n)$ worst case, $O(\log n)$ average
- Insertion: $O(n)$ worst case, $O(\log n)$ average
- Deletion: $O(n)$ worst case, $O(\log n)$ average

3.1.1 C++ Implementation

```
#include <iostream>
#include <queue>
using namespace std;

struct TreeNode {
    int data;
    TreeNode* left;
```

```

    TreeNode* right;

    TreeNode(int value) : data(value), left(nullptr), right(nullptr) {}
};

class BinaryTree {
private:
    TreeNode* root;

    void inorderHelper(TreeNode* node) {
        if (node) {
            inorderHelper(node->left);
            cout << node->data << " ";
            inorderHelper(node->right);
        }
    }

    void preorderHelper(TreeNode* node) {
        if (node) {
            cout << node->data << " ";
            preorderHelper(node->left);
            preorderHelper(node->right);
        }
    }

    void postorderHelper(TreeNode* node) {
        if (node) {
            postorderHelper(node->left);
            postorderHelper(node->right);
            cout << node->data << " ";
        }
    }

public:
    BinaryTree() : root(nullptr) {}

    void insert(int data) {
        TreeNode* newNode = new TreeNode(data);
        if (!root) {
            root = newNode;
            return;
        }
    }
};

```

```

    }

    queue<TreeNode*> q;
    q.push(root);

    while (!q.empty()) {
        TreeNode* current = q.front();
        q.pop();

        if (!current->left) {
            current->left = newNode;
            return;
        } else if (!current->right) {
            current->right = newNode;
            return;
        } else {
            q.push(current->left);
            q.push(current->right);
        }
    }
}

void inorder() {
    inorderHelper(root);
    cout << endl;
}

void preorder() {
    preorderHelper(root);
    cout << endl;
}

void postorder() {
    postorderHelper(root);
    cout << endl;
}

};

```

3.1.2 Python Implementation

```
from collections import deque

class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinaryTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = TreeNode(data)
        if not self.root:
            self.root = new_node
            return

        queue = deque([self.root])
        while queue:
            current = queue.popleft()

            if not current.left:
                current.left = new_node
                return
            elif not current.right:
                current.right = new_node
                return
            else:
                queue.append(current.left)
                queue.append(current.right)

    def inorder(self, node=None):
        if node is None:
            node = self.root

        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
```

```

        self.inorder(node.right)

def preorder(self, node=None):
    if node is None:
        node = self.root

    if node:
        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

def postorder(self, node=None):
    if node is None:
        node = self.root

    if node:
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")

```

3.2 Binary Search Tree (BST)

Definition

A binary tree where left child values are less than parent, and right child values are greater than parent.

Detailed Explanation: A Binary Search Tree is a special type of binary tree that maintains the BST property: for every node, all values in the left subtree are smaller than the node's value, and all values in the right subtree are greater. This property enables efficient searching, insertion, and deletion operations.

BST Property

- Left subtree contains only nodes with values less than root
- Right subtree contains only nodes with values greater than root
- Both left and right subtrees are also BSTs
- No duplicate values (in basic implementation)

Key Operations

- **Search:** Start from root, go left if target is smaller, right if larger
- **Insert:** Find correct position maintaining BST property
- **Delete:** Three cases - leaf node, node with one child, node with two children
- **Traversal:** Inorder traversal gives sorted sequence

Tree Balance

- **Balanced BST:** Height difference between subtrees ≤ 1
- **Skewed BST:** Degenerates into linked list (worst case)
- **Self-balancing BSTs:** AVL trees, Red-Black trees maintain balance

Key Characteristics

- Maintains sorted order
- Recursive structure
- In-order traversal gives sorted sequence
- Efficient for search operations

Advantages

- Efficient searching $O(\log n)$ average case
- Inorder traversal gives sorted data
- Dynamic size
- No need to shift elements like arrays
- Efficient insertion and deletion

Disadvantages

- Can become unbalanced (skewed)
- Worst case $O(n)$ operations
- Extra memory for pointers
- More complex than arrays
- No constant time access

Balancing Techniques

- **AVL Trees:** Height-balanced BST
- **Red-Black Trees:** Approximately balanced BST
- **Splay Trees:** Self-adjusting BST

Time Complexity

- Search: $O(\log n)$ average, $O(n)$ worst
- Insertion: $O(\log n)$ average, $O(n)$ worst
- Deletion: $O(\log n)$ average, $O(n)$ worst

3.2.1 C++ Implementation

```
#include <iostream>
using namespace std;

struct BSTNode {
    int data;
    BSTNode* left;
    BSTNode* right;

    BSTNode(int value) : data(value), left(nullptr), right(nullptr) {}
};

class BST {
private:
    BSTNode* root;

    BSTNode* insertHelper(BSTNode* node, int data) {
        if (!node) {
            return new BSTNode(data);
        }

        if (data < node->data) {
            node->left = insertHelper(node->left, data);
        } else if (data > node->data) {
            node->right = insertHelper(node->right, data);
        }

        return node;
    }

    BSTNode* searchHelper(BSTNode* node, int data) {
        if (!node || node->data == data) {
            return node;
        }
    }
};
```



```

    }

    if (data < node->data) {
        return searchHelper(node->left, data);
    }
    return searchHelper(node->right, data);
}

BSTNode* findMin(BSTNode* node) {
    while (node && node->left) {
        node = node->left;
    }
    return node;
}

BSTNode* deleteHelper(BSTNode* node, int data) {
    if (!node) return node;

    if (data < node->data) {
        node->left = deleteHelper(node->left, data);
    } else if (data > node->data) {
        node->right = deleteHelper(node->right, data);
    } else {
        if (!node->left) {
            BSTNode* temp = node->right;
            delete node;
            return temp;
        } else if (!node->right) {
            BSTNode* temp = node->left;
            delete node;
            return temp;
        }

        BSTNode* temp = findMin(node->right);
        node->data = temp->data;
        node->right = deleteHelper(node->right, temp->data);
    }
    return node;
}

void inorderHelper(BSTNode* node) {

```

```

        if (node) {
            inorderHelper(node->left);
            cout << node->data << " ";
            inorderHelper(node->right);
        }
    }

public:
    BST() : root(nullptr) {}

    void insert(int data) {
        root = insertHelper(root, data);
    }

    bool search(int data) {
        return searchHelper(root, data) != nullptr;
    }

    void remove(int data) {
        root = deleteHelper(root, data);
    }

    void inorder() {
        inorderHelper(root);
        cout << endl;
    }
};

```

3.2.2 Python Implementation

```

class BSTNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None

```

```

def insert(self, data):
    self.root = self._insert_helper(self.root, data)

def _insert_helper(self, node, data):
    if not node:
        return BSTNode(data)

    if data < node.data:
        node.left = self._insert_helper(node.left, data)
    elif data > node.data:
        node.right = self._insert_helper(node.right, data)

    return node

def search(self, data):
    return self._search_helper(self.root, data)

def _search_helper(self, node, data):
    if not node or node.data == data:
        return node is not None

    if data < node.data:
        return self._search_helper(node.left, data)
    return self._search_helper(node.right, data)

def delete(self, data):
    self.root = self._delete_helper(self.root, data)

def _delete_helper(self, node, data):
    if not node:
        return node

    if data < node.data:
        node.left = self._delete_helper(node.left, data)
    elif data > node.data:
        node.right = self._delete_helper(node.right, data)
    else:
        if not node.left:
            return node.right
        elif not node.right:
            return node.left

```

```

        temp = self._find_min(node.right)
        node.data = temp.data
        node.right = self._delete_helper(node.right, temp.data)

    return node

def _find_min(self, node):
    while node and node.left:
        node = node.left
    return node

def inorder(self, node=None):
    if node is None:
        node = self.root

    if node:
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)

```

3.3 Hash Table

Definition

A data structure that implements an associative array using a hash function to compute an index.

Detailed Explanation: A hash table (hash map) is a data structure that uses a hash function to map keys to values. It provides very fast average-case performance for insertion, deletion, and lookup operations. The hash function converts keys into array indices, allowing direct access to stored values.

How Hash Tables Work

1. **Hash Function:** Converts key into array index
2. **Bucket:** Array location where key-value pairs are stored
3. **Collision:** When multiple keys hash to same index
4. **Load Factor:** Ratio of stored elements to table size

Hash Functions

- **Division Method:** $\text{key} \% \text{table_size}$
- **Multiplication Method:** $\text{floor}(\text{table_size} * (\text{key} * A \bmod 1))$
- **Universal Hashing:** Random hash function selection

Collision Resolution

- **Chaining:** Store multiple elements in linked lists at each bucket
- **Open Addressing:** Find alternative locations within the table
 - Linear Probing: Check next slot sequentially
 - Quadratic Probing: Check slots at quadratic intervals
 - Double Hashing: Use second hash function

Key Characteristics

- Key-value pair storage
- Fast average-case operations
- Uses hash function for indexing
- May have collisions

Advantages

- Very fast average-case operations $O(1)$
- Efficient for large datasets
- Simple concept and implementation
- Good for caching and memoization
- Constant time complexity for basic operations

Disadvantages

- Worst-case $O(n)$ performance
- No ordering of elements
- Hash function dependency
- Memory overhead
- Collision handling complexity

Load Factor Impact

- **Low Load Factor** (< 0.5): Fast operations, memory waste
- **High Load Factor** (> 0.75): More collisions, slower operations
- **Optimal Range**: 0.5 - 0.75 for good performance

Time Complexity

- Average: $O(1)$ for search, insert, delete
- Worst: $O(n)$ for search, insert, delete

3.3.1 C++ Implementation

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

class HashTable {
private:
    vector<list<pair<int, int>>> table;
    int capacity;

    int hashFunction(int key) {
        return key % capacity;
    }

public:
    HashTable(int cap) : capacity(cap) {
        table.resize(capacity);
    }

    void insert(int key, int value) {
        int index = hashFunction(key);

        for (auto& pair : table[index]) {
            if (pair.first == key) {
                pair.second = value;
                return;
            }
        }
    }
};
```

```

    }

    table[index].push_back({key, value});
}

bool search(int key, int& value) {
    int index = hashFunction(key);

    for (const auto& pair : table[index]) {
        if (pair.first == key) {
            value = pair.second;
            return true;
        }
    }
    return false;
}

void remove(int key) {
    int index = hashFunction(key);

    table[index].remove_if([key](const pair<int, int>& p) {
        return p.first == key;
    });
}

void display() {
    for (int i = 0; i < capacity; i++) {
        cout << "Bucket " << i << ": ";
        for (const auto& pair : table[i]) {
            cout << "(" << pair.first << ", " << pair.second << ") ";
        }
        cout << endl;
    }
}

};

```

3.3.2 Python Implementation

```
class HashTable:
    def __init__(self, capacity=10):
        self.capacity = capacity
        self.table = [[] for _ in range(capacity)]

    def _hash_function(self, key):
        return key % self.capacity

    def insert(self, key, value):
        index = self._hash_function(key)

        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return

        self.table[index].append((key, value))

    def search(self, key):
        index = self._hash_function(key)

        for k, v in self.table[index]:
            if k == key:
                return v
        return None

    def remove(self, key):
        index = self._hash_function(key)

        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False

    def display(self):
        for i, bucket in enumerate(self.table):
            print(f"Bucket {i}: {bucket}")
```


3.4 Graph

Definition

A collection of vertices (nodes) connected by edges.

Detailed Explanation: A graph is a non-linear data structure consisting of vertices (nodes) and edges that connect these vertices. Graphs are used to represent relationships between entities and are fundamental in computer science for modeling networks, social connections, transportation systems, and many other real-world scenarios.

Graph Components

- **Vertex/Node:** Individual entity in the graph
- **Edge:** Connection between two vertices
- **Degree:** Number of edges connected to a vertex
- **Path:** Sequence of vertices connected by edges
- **Cycle:** Path that starts and ends at the same vertex

Types of Graphs

- **Directed vs Undirected:** Edges have direction or not
- **Weighted vs Unweighted:** Edges have weights/costs or not
- **Connected vs Disconnected:** All vertices reachable or not
- **Cyclic vs Acyclic:** Contains cycles or not
- **Simple vs Multi-graph:** At most one edge between vertices or multiple edges allowed

Graph Representations

1. **Adjacency Matrix:** 2D array where $\text{matrix}[i][j] = 1$ if edge exists
2. **Adjacency List:** Array of lists, each list contains neighbors
3. **Edge List:** List of all edges as pairs (u, v)

Graph Traversal Algorithms

- **Breadth-First Search (BFS):** Level-by-level exploration using queue
- **Depth-First Search (DFS):** Deep exploration using stack/recursion

Key Characteristics

- Non-linear structure
- Flexible connections between nodes
- Can represent complex relationships
- Multiple traversal methods

Advantages

- Natural representation of networks
- Flexible structure for complex relationships
- Supports various algorithms
- Models real-world problems effectively

Disadvantages

- Complex implementation compared to linear structures
- Memory overhead for storing edges
- Some operations can be expensive
- Requires careful handling of cycles

Common Graph Algorithms

- **Shortest Path:** Dijkstra's, Bellman-Ford, Floyd-Warshall
- **Minimum Spanning Tree:** Kruskal's, Prim's
- **Topological Sort:** For directed acyclic graphs
- **Strongly Connected Components:** Tarjan's, Kosaraju's

Space Complexity

- Adjacency Matrix: $O(V^2)$
- Adjacency List: $O(V + E)$ where V = vertices, E = edges

3.4.1 C++ Implementation (Adjacency List)

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
using namespace std;
```

```

class Graph {
private:
    int vertices;
    vector<vector<int>> adjList;

public:
    Graph(int v) : vertices(v) {
        adjList.resize(v);
    }

    void addEdge(int src, int dest) {
        adjList[src].push_back(dest);
        adjList[dest].push_back(src); // For undirected graph
    }

    void BFS(int start) {
        vector<bool> visited(vertices, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        while (!q.empty()) {
            int vertex = q.front();
            q.pop();
            cout << vertex << " ";

            for (int neighbor : adjList[vertex]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
        cout << endl;
    }

    void DFS(int start) {
        vector<bool> visited(vertices, false);
        stack<int> s;
    }

```

```

s.push(start);

while (!s.empty()) {
    int vertex = s.top();
    s.pop();

    if (!visited[vertex]) {
        visited[vertex] = true;
        cout << vertex << " ";

        for (int neighbor : adjList[vertex]) {
            if (!visited[neighbor]) {
                s.push(neighbor);
            }
        }
    }
    cout << endl;
}

void display() {
    for (int i = 0; i < vertices; i++) {
        cout << i << ": ";
        for (int neighbor : adjList[i]) {
            cout << neighbor << " ";
        }
        cout << endl;
    }
}
};

```

3.4.2 Python Implementation

```

from collections import deque, defaultdict

class Graph:
    def __init__(self, vertices):
        self.vertices = vertices
        self.adj_list = defaultdict(list)

```

```

def add_edge(self, src, dest):
    self.adj_list[src].append(dest)
    self.adj_list[dest].append(src) # For undirected graph

def bfs(self, start):
    visited = [False] * self.vertices
    queue = deque([start])
    visited[start] = True

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in self.adj_list[vertex]:
            if not visited[neighbor]:
                visited[neighbor] = True
                queue.append(neighbor)

    print()

def dfs(self, start):
    visited = [False] * self.vertices
    stack = [start]

    while stack:
        vertex = stack.pop()

        if not visited[vertex]:
            visited[vertex] = True
            print(vertex, end=" ")

            for neighbor in self.adj_list[vertex]:
                if not visited[neighbor]:
                    stack.append(neighbor)

    print()

def display(self):
    for vertex in range(self.vertices):
        print(f"{vertex}: {self.adj_list[vertex]}")

```

4 Sorting Algorithms

4.1 Bubble Sort

Definition

Repeatedly steps through the list, compares adjacent elements and swaps them if they're in wrong order.

Detailed Explanation: Bubble Sort is the simplest sorting algorithm that works by repeatedly stepping through the list, comparing adjacent elements and swapping them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The algorithm gets its name because smaller elements “bubble” to the top of the list.

How It Works

1. Compare adjacent elements
2. Swap if they're in wrong order
3. Continue through entire array
4. Repeat until no swaps needed
5. Each pass guarantees largest element reaches its correct position

Algorithm Steps

- Start with first two elements
- Compare and swap if necessary
- Move to next pair
- Continue until end of array
- Repeat entire process
- Stop when no swaps occur in a complete pass

Optimization

- **Early termination:** Stop if no swaps occur in a pass
- **Boundary reduction:** Each pass reduces the boundary as largest elements settle

Key Characteristics

- Comparison-based algorithm
- In-place sorting ($O(1)$ extra space)
- Stable sorting (maintains relative order of equal elements)
- Adaptive (performs better on nearly sorted arrays)

Advantages

- Simple implementation
- No additional memory space needed
- Stable sorting algorithm
- Can detect if list is already sorted
- Good for educational purposes

Disadvantages

- Poor time complexity $O(n^2)$
- Not suitable for large datasets
- More swaps compared to other algorithms
- Generally inefficient compared to other sorting methods

Variants

- **Cocktail Shaker Sort:** Bidirectional bubble sort
- **Odd-Even Sort:** Parallel version of bubble sort

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

4.1.1 C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;

void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
```

```

        swap(arr[j], arr[j + 1]);
        swapped = true;
    }
}
if (!swapped) break;
}
}

```

4.1.2 Python Implementation

```

def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr

```

4.2 Selection Sort

Definition

Finds the minimum element and places it at the beginning, then repeats for the remaining elements.

Detailed Explanation: Selection Sort divides the input list into two parts: a sorted portion at the left end and an unsorted portion at the right end. Initially, the sorted portion is empty and the unsorted portion is the entire list. The algorithm proceeds by finding the smallest element in the unsorted portion and swapping it with the first element of the unsorted portion.

How It Works

1. Find the minimum element in the unsorted portion
2. Swap it with the first element of unsorted portion
3. Move the boundary between sorted and unsorted portions

4. Repeat until entire array is sorted

Algorithm Steps

- Start with entire array as unsorted
- Find minimum element in unsorted portion
- Swap with first unsorted element
- Expand sorted portion by one element
- Repeat until array is completely sorted

Key Characteristics

- Comparison-based algorithm
- In-place sorting ($O(1)$ extra space)
- Not stable (may change relative order of equal elements)
- Not adaptive (always performs same number of comparisons)

Advantages

- Simple implementation
- In-place sorting algorithm
- Minimum number of swaps (at most $n-1$)
- Performance is consistent regardless of input
- Good for situations where swap cost is high

Disadvantages

- Poor time complexity $O(n^2)$
- Not stable sorting algorithm
- Not adaptive to input data
- Many unnecessary comparisons
- Not suitable for large datasets

Comparison with Bubble Sort

- Fewer swaps than bubble sort
- Same time complexity but more efficient in practice
- Not adaptive unlike optimized bubble sort

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

4.2.1 C++ Implementation

```
void selectionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        if (minIdx != i) {
            swap(arr[i], arr[minIdx]);
        }
    }
}
```

4.2.2 Python Implementation

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

4.3 Insertion Sort

Definition

Builds the sorted array one element at a time by inserting each element into its correct position.

Detailed Explanation: Insertion Sort builds the final sorted array one element at a time. It works by taking elements from the unsorted portion and inserting them into their correct position in the sorted portion. It's similar to how you might sort playing cards in your hands - pick up cards one by one and insert each into its proper position among the cards you've already sorted.

How It Works

1. Start with second element (assume first is sorted)
2. Compare current element with elements in sorted portion
3. Shift larger elements to the right
4. Insert current element in correct position
5. Repeat for all remaining elements

Algorithm Steps

- Assume first element is sorted
- Take next element as key
- Compare key with sorted elements from right to left
- Shift elements greater than key to the right
- Insert key in correct position
- Repeat for all elements

Key Characteristics

- Comparison-based algorithm
- In-place sorting ($O(1)$ extra space)
- Stable sorting (maintains relative order of equal elements)
- Adaptive (efficient for nearly sorted data)
- Online (can sort data as it's received)

Advantages

- Simple implementation
- Efficient for small datasets

- Adaptive algorithm ($O(n)$ best case)
- Stable sorting algorithm
- In-place algorithm
- Online algorithm
- More efficient than selection and bubble sort

Disadvantages

- Poor time complexity for large datasets $O(n^2)$
- More writes compared to selection sort
- Not efficient for large lists
- Quadratic time complexity in average and worst case

Best Case

Already sorted array $O(n)$

Average Case

Random order $O(n^2)$

Worst Case

Reverse sorted array $O(n^2)$

Variants

- **Binary Insertion Sort:** Uses binary search to find insertion position
- **Shell Sort:** Generalization of insertion sort

Time Complexity

$O(n^2)$ worst case, $O(n)$ best case

Space Complexity

$O(1)$

4.3.1 C++ Implementation

```
void insertionSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

4.3.2 Python Implementation

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

4.4 Merge Sort

Definition

Divide-and-conquer algorithm that divides the array into halves, sorts them separately, then merges them.

Detailed Explanation: Merge Sort is a divide-and-conquer algorithm that divides the input array into two halves, recursively sorts both halves, and then merges the two sorted halves. The key operation is the merge process, which combines two sorted arrays into one sorted array.

How It Works

1. **Divide:** Split array into two halves
2. **Conquer:** Recursively sort both halves
3. **Combine:** Merge the two sorted halves

Algorithm Steps

- If array has one element, it's already sorted (base case)
- Divide array into two halves
- Recursively sort left half
- Recursively sort right half
- Merge the two sorted halves

Merge Process

- Compare elements from both sorted arrays
- Select smaller element and add to result
- Continue until one array is exhausted
- Copy remaining elements from other array

Key Characteristics

- Divide-and-conquer approach
- Comparison-based algorithm
- Stable sorting (maintains relative order)
- Not in-place (requires extra space)
- Consistent performance regardless of input

Advantages

- Guaranteed $O(n \log n)$ time complexity
- Stable sorting algorithm
- Predictable performance
- Suitable for large datasets
- Can be easily parallelized
- External sorting capability

Disadvantages

- Requires $O(n)$ extra space
- Not in-place algorithm
- Slightly slower than quicksort in practice
- Recursive overhead
- Not adaptive (doesn't benefit from pre-sorted data)

Variants

- **Bottom-up Merge Sort:** Iterative version
- **Natural Merge Sort:** Takes advantage of existing runs
- **Multi-way Merge Sort:** Merges more than two arrays

Time Complexity

$O(n \log n)$

Space Complexity

$O(n)$

4.4.1 C++ Implementation

```
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> leftArr(n1), rightArr(n2);

    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        rightArr[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k++] = leftArr[i++];
        }
    }
}
```

```

        } else {
            arr[k++] = rightArr[j++];
        }
    }

    while (i < n1) arr[k++] = leftArr[i++];
    while (j < n2) arr[k++] = rightArr[j++];
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

```

4.4.2 Python Implementation

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result += left[i:] + right[j:]
    return result

```



```
j += 1

result.extend(left[i:])
result.extend(right[j:])
return result
```

4.5 Quick Sort

Definition

Picks a pivot element and partitions the array so elements smaller than pivot are on left, larger on right.

Detailed Explanation: Quick Sort is a divide-and-conquer algorithm that picks a pivot element and partitions the array around the pivot such that elements smaller than the pivot are on the left and elements greater than the pivot are on the right. It then recursively applies the same process to the sub-arrays.

How It Works

1. **Choose a pivot** element from the array
2. **Partition** the array so elements $<$ pivot are left, $>$ pivot are right
3. **Recursively apply** quicksort to left and right sub-arrays

Partitioning Process (Lomuto Scheme)

- Choose last element as pivot
- Maintain index of smaller element
- Traverse array and swap elements smaller than pivot
- Place pivot in correct position

Pivot Selection Strategies

- **First Element:** Simple but poor for sorted arrays
- **Last Element:** Most common in implementations
- **Random Element:** Good average performance
- **Median-of-Three:** First, middle, last elements
- **Median-of-Medians:** Guaranteed good pivot

Key Characteristics

- Divide-and-conquer approach
- In-place sorting (with careful implementation)
- Not stable (may change relative order)
- Comparison-based algorithm
- Performance depends on pivot selection

Advantages

- Excellent average-case performance $O(n \log n)$
- In-place sorting algorithm
- Cache-efficient due to good locality
- Widely used and well-optimized
- Generally faster than other $O(n \log n)$ algorithms

Disadvantages

- Worst-case $O(n^2)$ performance
- Not stable sorting algorithm
- Performance heavily depends on pivot selection
- Recursive overhead
- Poor performance on already sorted arrays (without optimizations)

Optimizations

- **Random Pivot:** Avoid worst-case on sorted data
- **Three-way Partitioning:** Handle duplicates efficiently
- **Hybrid Approach:** Switch to insertion sort for small arrays
- **Tail Recursion:** Reduce recursive calls

Time Complexity

$O(n \log n)$ average, $O(n^2)$ worst case

Space Complexity

$O(\log n)$

4.5.1 C++ Implementation

```
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

4.5.2 Python Implementation

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1

    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi - 1)
        quick_sort(arr, pi + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
```

```

for j in range(low, high):
    if arr[j] < pivot:
        i += 1
        arr[i], arr[j] = arr[j], arr[i]

arr[i + 1], arr[high] = arr[high], arr[i + 1]
return i + 1

```

4.6 Heap Sort

Definition

Uses a binary heap data structure to sort elements.

Detailed Explanation: Heap Sort is a comparison-based sorting algorithm that uses a binary heap data structure. It works by building a max heap from the input data, then repeatedly extracting the maximum element from the heap and placing it at the end of the sorted array. The heap property ensures that the largest element is always at the root.

Binary Heap Properties

- **Complete Binary Tree:** All levels filled except possibly the last
- **Heap Property:**
 - Max Heap: Parent $\geq$ children
 - Min Heap: Parent $\leq$ children
- **Array Representation:** For node at index i :
 - Left child: $2*i + 1$
 - Right child: $2*i + 2$
 - Parent: $(i-1)/2$

How It Works

1. **Build Max Heap:** Convert array into max heap
2. **Extract Maximum:** Swap root with last element
3. **Heapify:** Restore heap property for reduced heap
4. **Repeat:** Until heap size becomes 1

Heapify Process

- Compare parent with children
- Swap parent with larger child if necessary
- Recursively heapify affected subtree
- Continue until heap property is restored

Algorithm Steps

- Start from last non-leaf node
- Apply heapify to build initial max heap
- Repeatedly extract maximum and heapify
- Result is sorted array

Key Characteristics

- Comparison-based algorithm
- In-place sorting algorithm
- Not stable (may change relative order)
- Consistent $O(n \log n)$ performance
- Uses binary heap data structure

Advantages

- Guaranteed $O(n \log n)$ time complexity
- In-place sorting algorithm
- Not recursive (iterative implementation possible)
- Consistent performance regardless of input
- Memory efficient

Disadvantages

- Not stable sorting algorithm
- Poor cache locality
- Not adaptive (doesn't benefit from pre-sorted data)
- Slower than quicksort in practice
- Complex implementation compared to simple sorts

Variants

- **Bottom-up Heap Construction:** More efficient heap building
- **Smoothsort:** Adaptive heap sort variant
- **Weak Heap Sort:** Uses weak heap data structure

Time Complexity

$O(n \log n)$

Space Complexity

$O(1)$

4.6.1 C++ Implementation

```
void heapify(vector<int>& arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
```

```

        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

```

4.6.2 Python Implementation

```

def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left

    if right < n and arr[right] > arr[largest]:
        largest = right

    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)

    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)

    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)

    return arr

```

5 Searching Algorithms

5.1 Linear Search

Definition

Sequentially checks each element until the target is found.

Detailed Explanation: Linear Search, also known as sequential search, is the simplest searching algorithm that checks every element in the list sequentially until the target element is found or the end of the list is reached. It works on both sorted and unsorted arrays and doesn't require any special data structure.

How It Works

1. Start from the first element
2. Compare current element with target
3. If match found, return index
4. If not found, move to next element
5. Continue until element found or end reached

Algorithm Steps

- Initialize index to 0
- Compare array[index] with target
- If equal, return index
- If not equal, increment index
- Repeat until found or index exceeds array size
- Return -1 if not found

Key Characteristics

- Simple and straightforward algorithm
- Works on unsorted data
- Sequential access pattern
- No preprocessing required
- Can be used on any data structure

Advantages

- Simple to implement and understand
- Works on unsorted arrays

- No additional space required
- Can find all occurrences easily
- Works on any data structure (arrays, linked lists)
- No preprocessing overhead

Disadvantages

- Poor time complexity $O(n)$
- Inefficient for large datasets
- Slow compared to other search algorithms
- Not suitable for frequently searched data
- Doesn't take advantage of sorted data

Variants

- **Sentinel Linear Search:** Adds target at end to eliminate boundary check
- **Transposition:** Moves found element one position forward
- **Move to Front:** Moves found element to beginning

Time Complexity

$O(n)$

Space Complexity

$O(1)$

5.1.1 C++ Implementation

```
int linearSearch(const vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
```

5.1.2 Python Implementation

```
def linear_search(arr, target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

5.2 Binary Search

Definition

Searches a sorted array by repeatedly dividing the search interval in half.

Detailed Explanation: Binary Search is an efficient algorithm for searching a sorted array by repeatedly dividing the search interval in half. It compares the target value to the middle element of the array and eliminates half of the remaining elements based on the comparison result. This divide-and-conquer approach makes it much faster than linear search.

Prerequisites

- Array must be sorted
- Random access to elements (works best with arrays)

How It Works

1. **Initialize** left and right pointers
2. **Calculate** middle index
3. **Compare** middle element with target
4. **Eliminate** half of the search space
5. **Repeat** until element found or search space exhausted

Algorithm Steps

- Set left = 0, right = array.length - 1
- While left ≤ right:
 - Calculate mid = (left + right) / 2
 - If array[mid] == target, return mid
 - If array[mid] < target, left = mid + 1
 - If array[mid] > target, right = mid - 1

- Return -1 if not found

Search Space Reduction

- Each comparison eliminates half of remaining elements
- Search space: $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \dots \rightarrow 1$
- Maximum comparisons: $\log(n)$

Key Characteristics

- Requires sorted data
- Divide-and-conquer approach
- Logarithmic time complexity
- Random access requirement
- Comparison-based algorithm

Advantages

- Excellent time complexity $O(\log n)$
- Much faster than linear search for large datasets
- Simple to implement
- Space efficient $O(1)$ for iterative version
- Widely applicable

Disadvantages

- Requires sorted data
- Only works with random access structures
- Preprocessing overhead (sorting)
- Not suitable for frequently changing data
- Overhead may not be worth it for small datasets

Implementation Variations

- **Iterative:** Uses loops, $O(1)$ space
- **Recursive:** Uses function calls, $O(\log n)$ space
- **Left-most/Right-most:** Find first/last occurrence

Common Pitfalls

- Integer overflow in mid calculation: use $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$
- Off-by-one errors in boundary conditions
- Assuming array is sorted

Extensions

- **Binary Search on Answer:** Search for optimal value in range
- **Exponential Search:** Find range then apply binary search
- **Interpolation Search:** Estimate position based on values

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$ iterative, $O(\log n)$ recursive

5.2.1 C++ Implementation

```
int binarySearch(const vector<int>& arr, int target) {
    int left = 0, right = arr.size() - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}

// Recursive version
```

```

int binarySearchRecursive(const vector<int>& arr, int target, int left, int right) {
    if (left > right) return -1;

    int mid = left + (right - left) / 2;

    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) return binarySearchRecursive(arr, target, mid + 1, right);
    else return binarySearchRecursive(arr, target, left, mid - 1);
}

```

5.2.2 Python Implementation

```

def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1

# Recursive version
def binary_search_recursive(arr, target, left=0, right=None):
    if right is None:
        right = len(arr) - 1

    if left > right:
        return -1

    mid = (left + right) // 2

    if arr[mid] == target:
        return mid
    elif arr[mid] < target:

```

```
        return binary_search_recursive(arr, target, mid + 1, right)
    else:
        return binary_search_recursive(arr, target, left, mid - 1)
```

6 Graph Algorithms

6.1 Breadth-First Search (BFS)

Definition

Graph traversal algorithm that explores vertices level by level, visiting all neighbors before moving to next level.

Detailed Explanation: BFS is a graph traversal algorithm that systematically explores vertices of a graph in breadth-first manner. It starts from a source vertex and explores all vertices at distance 1, then all vertices at distance 2, and so on. BFS uses a queue data structure to keep track of vertices to be explored.

How It Works

1. Start from source vertex
2. Mark source as visited and enqueue it
3. While queue is not empty:
 - Dequeue a vertex
 - Visit all unvisited neighbors
 - Mark neighbors as visited and enqueue them

Key Properties

- Explores vertices level by level
- Uses queue (FIFO) data structure
- Guarantees shortest path in unweighted graphs
- Systematic exploration pattern

Advantages

- Finds shortest path in unweighted graphs
- Complete algorithm (finds solution if exists)
- Optimal for exploring immediate neighbors

- Good for finding minimum spanning tree

Disadvantages

- High memory usage (stores all frontier vertices)
- Not suitable for large graphs
- Doesn't work well with weighted graphs
- Can be slow for deep searches

Time Complexity

$O(V + E)$ where V = vertices, E = edges

Space Complexity

$O(V)$ for the queue

6.1.1 C++ Implementation

```
void BFS(vector<vector<int>>& graph, int start) {
    int n = graph.size();
    vector<bool> visited(n, false);
    queue<int> q;

    visited[start] = true;
    q.push(start);

    while (!q.empty()) {
        int vertex = q.front();
        q.pop();
        cout << vertex << " ";

        for (int neighbor : graph[vertex]) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
```

```
    cout << endl;
}
```

6.1.2 Python Implementation

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    print()
```

6.2 Depth-First Search (DFS)

Definition

Graph traversal algorithm that explores as deep as possible before backtracking.

Detailed Explanation: DFS is a graph traversal algorithm that explores vertices by going as deep as possible along each branch before backtracking. It uses a stack (either explicit or through recursion) to remember vertices to backtrack to when a dead end is reached.

How It Works

1. Start from source vertex
2. Mark source as visited
3. For each unvisited neighbor:
 - Recursively apply DFS
4. Backtrack when no unvisited neighbors

Key Properties

- Explores as deep as possible
- Uses stack (LIFO) data structure or recursion
- May not find shortest path
- Memory efficient compared to BFS

Advantages

- Memory efficient ($O(\text{depth})$ space)
- Simple recursive implementation
- Good for exploring complete paths
- Useful for backtracking problems

Disadvantages

- May get stuck in infinite loops (cycles)
- Doesn't guarantee shortest path
- Stack overflow risk in deep recursion
- May explore unnecessary deep paths

Time Complexity

$O(V + E)$

Space Complexity

$O(V)$ in worst case for recursion stack

6.2.1 C++ Implementation

```
void DFSUtil(vector<vector<int>>& graph, int vertex, vector<bool>& visited) {
    visited[vertex] = true;
    cout << vertex << " ";

    for (int neighbor : graph[vertex]) {
        if (!visited[neighbor]) {
            DFSUtil(graph, neighbor, visited);
        }
    }
}
```

```

}

void DFS(vector<vector<int>>& graph, int start) {
    int n = graph.size();
    vector<bool> visited(n, false);
    DFSUtil(graph, start, visited);
    cout << endl;
}

// Iterative version
void DFSIterative(vector<vector<int>>& graph, int start) {
    int n = graph.size();
    vector<bool> visited(n, false);
    stack<int> s;

    s.push(start);

    while (!s.empty()) {
        int vertex = s.top();
        s.pop();

        if (!visited[vertex]) {
            visited[vertex] = true;
            cout << vertex << " ";

            for (int neighbor : graph[vertex]) {
                if (!visited[neighbor]) {
                    s.push(neighbor);
                }
            }
        }
    }
    cout << endl;
}

```

6.2.2 Python Implementation

```
def dfs_recursive(graph, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)
    print(start, end=" ")

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs_recursive(graph, neighbor, visited)

def dfs_iterative(graph, start):
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()

        if vertex not in visited:
            visited.add(vertex)
            print(vertex, end=" ")

            for neighbor in graph[vertex]:
                if neighbor not in visited:
                    stack.append(neighbor)

    print()
```

6.3 Dijkstra's Algorithm

Definition

Finds shortest paths from source vertex to all other vertices in weighted graph with non-negative weights.

Detailed Explanation: Dijkstra's algorithm solves the single-source shortest path problem for graphs with non-negative edge weights. It maintains a set of vertices whose shortest distance from source is known and gradually expands this set until all vertices are included.

How It Works

1. Initialize distances: source = 0, others = infinity
2. Create priority queue with all vertices
3. While queue not empty:
 - Extract vertex with minimum distance
 - Update distances to its neighbors
 - Relax edges if shorter path found

Key Concepts

- **Relaxation:** Update distance if shorter path found
- **Priority Queue:** Efficiently get vertex with minimum distance
- **Greedy Choice:** Always pick closest unvisited vertex

Time Complexity

$O((V + E) \log V)$ with binary heap

Space Complexity

$O(V)$

6.3.1 C++ Implementation

```
#include <vector>
#include <queue>
#include <limits>
using namespace std;

vector<int> dijkstra(vector<vector<pair<int, int>>>& graph, int start) {
    int n = graph.size();
    vector<int> dist(n, INT_MAX);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> pq;

    dist[start] = 0;
    pq.push({0, start});

    while (!pq.empty()) {
```

```

    int u = pq.top().second;
    pq.pop();

    for (auto& edge : graph[u]) {
        int v = edge.first;
        int weight = edge.second;

        if (dist[u] + weight < dist[v]) {
            dist[v] = dist[u] + weight;
            pq.push({dist[v], v});
        }
    }
}

return dist;
}

```

6.3.2 Python Implementation

```

import heapq

def dijkstra(graph, start):
    distances = {vertex: float('infinity') for vertex in graph}
    distances[start] = 0
    pq = [(0, start)]

    while pq:
        current_distance, current_vertex = heapq.heappop(pq)

        if current_distance > distances[current_vertex]:
            continue

        for neighbor, weight in graph[current_vertex]:
            distance = current_distance + weight

            if distance < distances[neighbor]:
                distances[neighbor] = distance
                heapq.heappush(pq, (distance, neighbor))

    return distances

```

7 Dynamic Programming

7.1 Dynamic Programming Fundamentals

Definition

Solving complex problems by breaking them into simpler subproblems and storing results to avoid redundant calculations.

Detailed Explanation: Dynamic Programming (DP) is an optimization technique that solves complex problems by breaking them down into simpler subproblems. It stores the results of subproblems to avoid computing the same results again. DP is applicable when the problem has optimal substructure and overlapping subproblems.

Key Principles

1. **Optimal Substructure:** Optimal solution contains optimal solutions to subproblems
2. **Overlapping Subproblems:** Same subproblems are solved multiple times
3. **Memoization:** Store results to avoid recomputation

DP Approaches

- **Top-down (Memoization):** Recursive with caching
- **Bottom-up (Tabulation):** Iterative table filling

Classic DP Problems

- **Fibonacci Numbers:** $F(n) = F(n-1) + F(n-2)$
- **Longest Common Subsequence:** Finding common subsequences
- **0/1 Knapsack:** Optimal item selection with weight constraint
- **Coin Change:** Minimum coins needed for amount
- **Edit Distance:** Minimum operations to transform strings

7.2 Fibonacci with DP

7.2.1 C++ Implementation

```
// Memoization (Top-down)
int fibMemo(int n, vector<int>& memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    memo[n] = fibMemo(n-1, memo) + fibMemo(n-2, memo);
    return memo[n];
}

// Tabulation (Bottom-up)
int fibDP(int n) {
    if (n <= 1) return n;
    vector<int> dp(n+1);
    dp[0] = 0; dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i-1] + dp[i-2];
    }
    return dp[n];
}

// Space-optimized
int fibOptimized(int n) {
    if (n <= 1) return n;
    int prev2 = 0, prev1 = 1, current;
    for (int i = 2; i <= n; i++) {
        current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return current;
}
```

7.2.2 Python Implementation

```
# Memoization
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]

# Tabulation
def fib_dp(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# Space-optimized
def fib_optimized(n):
    if n <= 1:
        return n
    prev2, prev1 = 0, 1
    for i in range(2, n + 1):
        current = prev1 + prev2
        prev2, prev1 = prev1, current
    return prev1
```


8 String Algorithms

8.1 KMP Algorithm (Pattern Matching)

Definition

Efficient string matching algorithm that uses preprocessing to avoid redundant character comparisons.

Detailed Explanation: The Knuth-Morris-Pratt (KMP) algorithm is an efficient string matching algorithm that finds occurrences of a pattern within a text. It preprocesses the pattern to create a failure function that helps skip characters during matching, avoiding redundant comparisons.

Key Concepts

- **Failure Function:** Longest proper prefix which is also suffix
- **No Backtracking:** Never moves backward in text
- **Linear Time:** $O(n + m)$ where n = text length, m = pattern length

How It Works

1. **Preprocessing:** Build failure function for pattern
2. **Matching:** Use failure function to skip characters efficiently
3. **Optimization:** Avoid redundant character comparisons

Time Complexity

$O(n + m)$

Space Complexity

$O(m)$

8.1.1 C++ Implementation

```
vector<int> computeLPS(string pattern) {  
    int m = pattern.length();  
    vector<int> lps(m, 0);  
    int len = 0;
```

```

    int i = 1;

    while (i < m) {
        if (pattern[i] == pattern[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }
    return lps;
}

vector<int> KMPSearch(string text, string pattern) {
    vector<int> result;
    int n = text.length();
    int m = pattern.length();

    vector<int> lps = computeLPS(pattern);

    int i = 0; // index for text
    int j = 0; // index for pattern

    while (i < n) {
        if (pattern[j] == text[i]) {
            i++;
            j++;
        }

        if (j == m) {
            result.push_back(i - j);
            j = lps[j - 1];
        } else if (i < n && pattern[j] != text[i]) {
            if (j != 0) {
                j = lps[j - 1];
            }
        }
    }
}

```

```

        } else {
            i++;
        }
    }
}
return result;
}

```

8.1.2 Python Implementation

```

def compute_lps(pattern):
    m = len(pattern)
    lps = [0] * m
    length = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length - 1]
            else:
                lps[i] = 0
                i += 1
    return lps

def kmp_search(text, pattern):
    n = len(text)
    m = len(pattern)

    lps = compute_lps(pattern)
    result = []

    i = j = 0

    while i < n:
        if pattern[j] == text[i]:

```

```

        i += 1
        j += 1

    if j == m:
        result.append(i - j)
        j = lps[j - 1]
    elif i < n and pattern[j] != text[i]:
        if j != 0:
            j = lps[j - 1]
        else:
            i += 1

    return result

```

9 Advanced Data Structures

9.1 Trie (Prefix Tree)

Definition

Tree-like data structure used to store and search strings efficiently, especially for prefix-based operations.

Detailed Explanation: A Trie is a tree data structure used for storing and searching strings efficiently. Each node represents a character, and paths from root to leaves represent complete strings. It's particularly useful for prefix-based operations like autocomplete and spell checkers.

Key Properties

- Root represents empty string
- Each edge represents a character
- Nodes can be marked as end-of-word
- Common prefixes share same path

Advantages

- Efficient prefix searches $O(m)$ where m = key length
- Space efficient for large sets with common prefixes
- Easy to implement prefix-based operations

- Supports dynamic insertions and deletions

Disadvantages

- Space overhead for storing pointers
- Poor cache locality
- Memory intensive for small datasets
- Complex implementation compared to hash tables

Time Complexity

Insert/Search/Delete: $O(m)$ where m = key length

Space Complexity

$O(\text{ALPHABET_SIZE} * N * M)$ worst case

9.1.1 C++ Implementation

```
class TrieNode {
public:
    TrieNode* children[26];
    bool isEndOfWord;

    TrieNode() {
        isEndOfWord = false;
        for (int i = 0; i < 26; i++) {
            children[i] = nullptr;
        }
    }
};

class Trie {
private:
    TrieNode* root;

public:
    Trie() {
        root = new TrieNode();
    }
};
```

```

    }

    void insert(string word) {
        TrieNode* current = root;
        for (char c : word) {
            int index = c - 'a';
            if (current->children[index] == nullptr) {
                current->children[index] = new TrieNode();
            }
            current = current->children[index];
        }
        current->isEndOfWord = true;
    }

    bool search(string word) {
        TrieNode* current = root;
        for (char c : word) {
            int index = c - 'a';
            if (current->children[index] == nullptr) {
                return false;
            }
            current = current->children[index];
        }
        return current->isEndOfWord;
    }

    bool startsWith(string prefix) {
        TrieNode* current = root;
        for (char c : prefix) {
            int index = c - 'a';
            if (current->children[index] == nullptr) {
                return false;
            }
            current = current->children[index];
        }
        return true;
    }
};

```

9.1.2 Python Implementation

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        current = self.root
        for char in word:
            if char not in current.children:
                current.children[char] = TrieNode()
            current = current.children[char]
        current.is_end_of_word = True

    def search(self, word):
        current = self.root
        for char in word:
            if char not in current.children:
                return False
            current = current.children[char]
        return current.is_end_of_word

    def starts_with(self, prefix):
        current = self.root
        for char in prefix:
            if char not in current.children:
                return False
            current = current.children[char]
        return True
```

9.2 Segment Tree

Definition

Tree data structure for range queries and updates on arrays.

Detailed Explanation: A Segment Tree is a binary tree used for range queries on arrays. Each node stores information about a segment of the array, allowing efficient range queries and updates. It's particularly useful when you need to perform many range sum, minimum, maximum, or other associative operations.

Key Properties

- Binary tree with array elements at leaves
- Internal nodes store aggregate information
- Height is $O(\log n)$
- Supports both range queries and updates

Time Complexity

- Build: $O(n)$
- Query/Update: $O(\log n)$

Space Complexity

$O(n)$

9.2.1 C++ Implementation

```
class SegmentTree {
private:
    vector<int> tree;
    int n;

    void build(vector<int>& arr, int node, int start, int end) {
        if (start == end) {
            tree[node] = arr[start];
        } else {
            int mid = (start + end) / 2;
            build(arr, 2*node, start, mid);
```



```

        build(arr, 2*node+1, mid+1, end);
        tree[node] = tree[2*node] + tree[2*node+1];
    }
}

void updateHelper(int node, int start, int end, int idx, int val) {
    if (start == end) {
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;
        if (idx <= mid) {
            updateHelper(2*node, start, mid, idx, val);
        } else {
            updateHelper(2*node+1, mid+1, end, idx, val);
        }
        tree[node] = tree[2*node] + tree[2*node+1];
    }
}

int queryHelper(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return 0;
    }
    if (l <= start && end <= r) {
        return tree[node];
    }
    int mid = (start + end) / 2;
    int left_sum = queryHelper(2*node, start, mid, l, r);
    int right_sum = queryHelper(2*node+1, mid+1, end, l, r);
    return left_sum + right_sum;
}

public:
    SegmentTree(vector<int>& arr) {
        n = arr.size();
        tree.resize(4 * n);
        build(arr, 1, 0, n-1);
    }

    void update(int idx, int val) {
        updateHelper(1, 0, n-1, idx, val);
    }

```

```

    }

    int query(int l, int r) {
        return queryHelper(1, 0, n-1, l, r);
    }
};

```

9.2.2 Python Implementation

```

class SegmentTree:
    def __init__(self, arr):
        self.n = len(arr)
        self.tree = [0] * (4 * self.n)
        self.build(arr, 1, 0, self.n - 1)

    def build(self, arr, node, start, end):
        if start == end:
            self.tree[node] = arr[start]
        else:
            mid = (start + end) // 2
            self.build(arr, 2 * node, start, mid)
            self.build(arr, 2 * node + 1, mid + 1, end)
            self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def update(self, idx, val):
        self._update_helper(1, 0, self.n - 1, idx, val)

    def _update_helper(self, node, start, end, idx, val):
        if start == end:
            self.tree[node] = val
        else:
            mid = (start + end) // 2
            if idx <= mid:
                self._update_helper(2 * node, start, mid, idx, val)
            else:
                self._update_helper(2 * node + 1, mid + 1, end, idx, val)
            self.tree[node] = self.tree[2 * node] + self.tree[2 * node + 1]

    def query(self, l, r):
        return self._query_helper(1, 0, self.n - 1, l, r)

```

```

def _query_helper(self, node, start, end, l, r):
    if r < start or end < l:
        return 0
    if l <= start and end <= r:
        return self.tree[node]
    mid = (start + end) // 2
    left_sum = self._query_helper(2 * node, start, mid, l, r)
    right_sum = self._query_helper(2 * node + 1, mid + 1, end, l, r)
    return left_sum + right_sum

```

10 Conclusion

This comprehensive guide covers the fundamental data structures and algorithms essential for computer science and software development. Each data structure and algorithm has been explained with:

- Detailed conceptual explanations
- Advantages and disadvantages
- Time and space complexity analysis
- Complete implementations in both C++ and Python
- Key characteristics and properties

Understanding these concepts is crucial for: - Technical interviews - Algorithm design - Performance optimization - Problem-solving skills - Software architecture decisions

The choice of data structure or algorithm depends on the specific requirements of your problem, including time constraints, space limitations, and the operations you need to perform most frequently.